

MANIPAL UNIVERSITY JAIPUR · DSE 4150 · 7th Semester

Natural Language Processing

Illustrated Mid-Term Notes — Lectures 1 to 20

Intro & Ambiguity

DFA / NFA

Regular Expressions

Morphology

FSTs

Tokenization & BPE

Edit Distance

Exam	Mid-Term · Closed book · 30 marks
Built from	All 14 course PDFs (handout, notes, PPT-1/2/3, DFA/NFA decks, RE & stop-word decks, Lecture-20 notes)
How to use	Read the yellow “Exam” boxes first, then practise every worked example. Last page = one-page revision sheet.

Contents

1 · Introduction to NLP — Lec 2–5	3
2 · Formal Languages & Theory of Computation — Lec 6–7	5
3 · Finite Automata: DFA & NFA — Lec 6–10	6
3B · DFA Design Gallery & NFA→DFA — Lec 8–10	8
4 · Regular Expressions — Lec 10	11
5 · Morphology — Lec 11–13, 19	13
6 · Finite-State Morphological Parsing & FSTs — Lec 14–18	16
7 · Text Processing & Tokenization — Lec 20	21
8 · Byte Pair Encoding (BPE) — Lec 20	25
9 · Spelling Errors & Minimum Edit Distance — Lec 20 notes	28
10 · Likely Exam Questions	31
11 · One-Page Revision Sheet	32

SYLLABUS MAP (from the Course Handout)

Lec	Major topic	Topics
2–5	Introduction	Phases of NLP · Knowledge in speech & language processing · Ambiguity · Models & algorithms
6–10	Finite State Machines	Acceptor vs transducer · Formal language · DFA · NFA · DFA vs NFA · Basic regex patterns
11–19	Morphological Analysis	English morphology · Inflectional · Derivational · FS morphological parsing · FS lexicon · FSTs · Orthographic rules · Lexicon-free FSTs
20	Text Processing	Words & sentence tokenization (+ spelling / edit distance from the Lecture-20 notes)

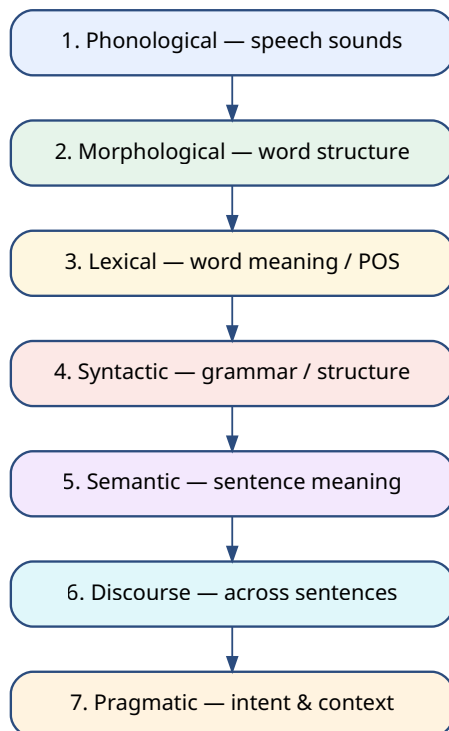
1 Introduction to NLP

Lec 2–5

Natural Language Processing (NLP) is the branch of Artificial Intelligence that enables computers to **understand, interpret and generate** human language (text or speech). It combines **computer science + linguistics + machine learning**.

Why it is hard: human language is *complex, ambiguous and context-dependent*. NLP bridges human communication and machine understanding.

Levels (phases) of NLP



Each level builds on the one above it.

With examples

Level	Example
Phonological	/b/ vs /p/ in speech
Morphological	un + happy; play → playing
Lexical	<i>bank</i> = river / money
Syntactic	"She is reading a book" ✓ vs "She reading is book" ✗
Semantic	"I saw a man with a telescope"
Discourse	"Ravi went home. He was tired."
Pragmatic	"Can you open the window?" = a request

EXAM

The 5 classic **phases**: Lexical → Syntactic → Semantic → Discourse integration → Pragmatic. Give one example each.

Ambiguity — the central problem

Type	Meaning	Example
Lexical	a word has several meanings (polysemy / homonymy)	<i>bank, bat; book</i> (N or V)
Syntactic / structural	a sentence has more than one parse	"I saw the man <u>with a telescope</u> "
Morphological	a word splits into morphemes in more than one way	<i>unionised; shorts vs short+s</i>
Semantic	the meaning of the whole is unclear	"Every student read a book" (same book?)
Referential / discourse	unclear what a pronoun refers to	"Ravi told Amit he passed."
Pragmatic	literal meaning ≠ intended meaning	"Can you pass the salt?"

How ambiguity is resolved: context (previous sentences, topic) · semantic analysis (word-sense disambiguation, semantic role labelling) · syntactic analysis · pragmatic analysis (speaker intent) · statistical / ML models (choose the most probable reading).

History of NLP

Era	Approach	Highlights
1950s–1970s	Rule-based	Georgetown–IBM experiment (1954) translated 60 Russian sentences into English; struggled with idioms and context
1980s–1990s	Statistical	n-grams, Hidden Markov Models, NER, statistical machine translation
2000s–today	Deep learning	Word2Vec, GloVe, RNN/LSTM, Transformers, BERT, GPT

Challenges of NLP

1. Language differences / diversity
2. Training data (quality, annotation)
3. Development time & resources (GPUs)
4. Phrasing ambiguities
5. Misspellings & grammatical errors
6. Innate bias in data/models
7. Words with multiple meanings
8. Multilingualism
9. Uncertainty & false positives
10. Continuous conversation (context tracking)

Applications

- Machine translation
- Information retrieval & search
- Question answering, chatbots, voice assistants
- Information extraction, NER, entity linking
- Summarization
- Text classification, spam filtering
- Sentiment analysis / opinion mining
- Spell & grammar checking
- Speech recognition & text-to-speech

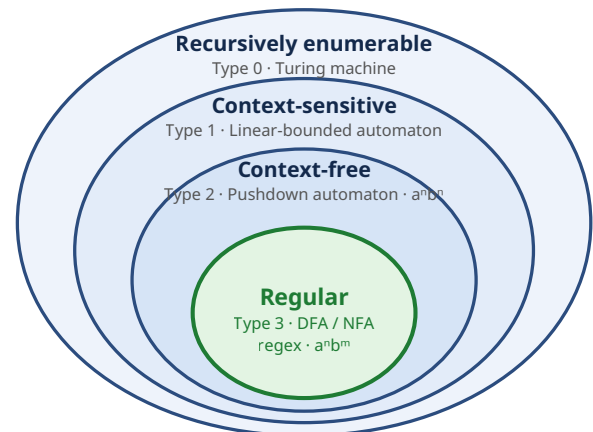
Grammars in NLP (short notes)

Grammar	Key idea
Transformational (Chomsky)	Deep structure (meaning) → <i>transformational rules</i> → surface structure. Active ↔ passive.
Lexical Functional (Bresnan & Kaplan)	c-structure (phrase/word order) + f-structure (subject, object, tense). No transformations.
Government & Binding (Chomsky)	Government: head ↔ complement (case). Binding: pronoun reference ("Ravi said he ...").
GPSG (Gazdar)	Non-transformational phrase-structure rules + features. $S \rightarrow NP VP$.
Dependency	Words depend on a head word (verb is root): boy → reads ← book.
Paninian	For Sanskrit/Indian free word-order languages. Karaka roles: Karta (doer), Karma (object), Karana (instrument), Adhikarana (location).
Tree-Adjoining (A. Joshi)	Initial + auxiliary trees, combined by <i>substitution</i> and <i>adjunction</i> .
Context-Free (CFG)	4 parts: non-terminals, terminals, productions, start symbol S.
Probabilistic CFG	Each rule has a probability; rules with the same LHS sum to 1; pick the most probable parse.

2 Formal Languages & Theory of Computation

Lec 6–7

Term	Definition
Symbol	smallest building block (a letter, digit, picture)
Alphabet Σ	finite, non-empty set of symbols, e.g. $\Sigma = \{a, b\}$
String w	finite sequence of symbols; $ w = \text{length}$
Empty string ϵ	zero symbols, $ \epsilon = 0$
Σ^* (Kleene star)	all strings over Σ including ϵ
Σ^+ (positive closure)	all strings except ϵ ; $\Sigma^+ = \{\epsilon\} \cup \Sigma^*$
Language L	any subset of Σ^* ; can be finite or infinite



The Chomsky hierarchy: each class contains the ones inside it.

COUNTING STRINGS

Over $\Sigma = \{a, b\}$, number of strings of length $n = 2^n$ (generally $|\Sigma|^n$).
Length 2 $\rightarrow$ aa, ab, ba, bb = 4.

Examples: $g^* = \{\epsilon, g, gg, ggg, \dots\}$; $g^+ = \{g, gg, \dots\}$.

Finite language: $L = \{xx, xy, yx, yy\}$. Infinite: all strings starting with b.

Language class	Grammar	Machine	Example
Regular	Regular grammar / regex	Finite automaton (DFA/ NFA)	a^n , a^nb^m , "ends in ab"
Context-free	CFG	Pushdown automaton	a^nb^n — no FSA can accept it
Context-sensitive	CSG	Linear-bounded automaton	$a^nb^nc^n$
Recursively enumerable	Unrestricted	Turing machine	—

ACCEPTOR vs TRANSDUCER (Lec 6)

Acceptor (FSA): reads a string and answers *accept / reject*, i.e. it *recognises* a language.

Transducer (FST): reads an input string and *writes an output string*, i.e. it defines a *relation* between two languages ($\text{cats} \leftrightarrow \text{cat} + \text{N} + \text{PL}$).

HOW AN AUTOMATON WORKS

1. Start in q_0 .
2. Read one symbol and move by δ .
3. Repeat until the input ends.
4. Accept if the current state $\in F$, otherwise reject. A missing transition also means reject.

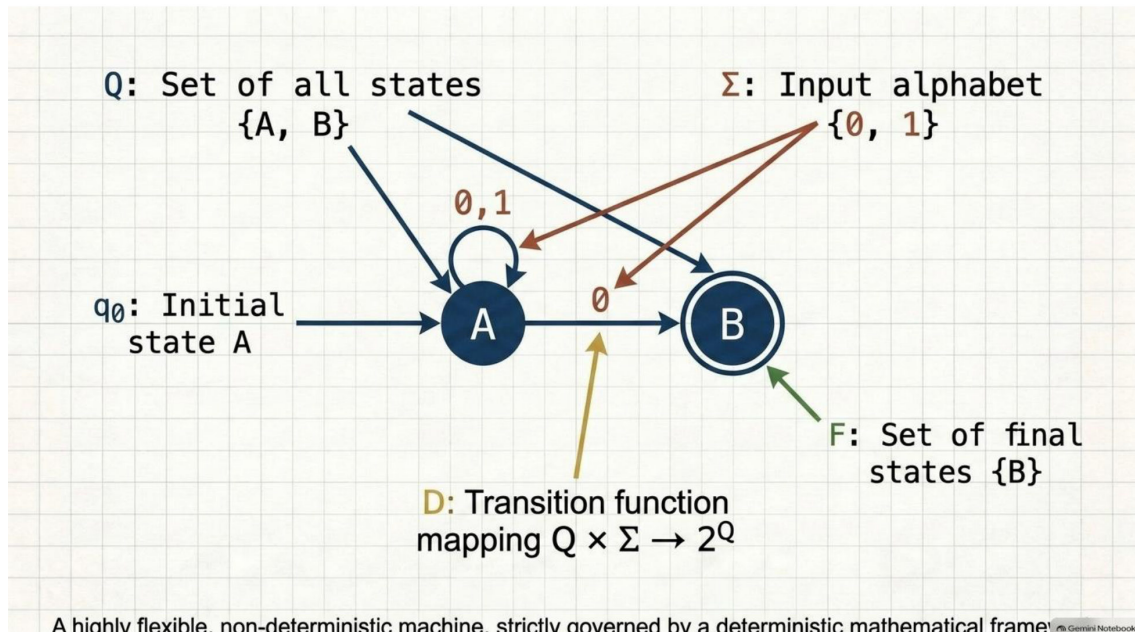
3 Finite Automata: DFA & NFA

Lec 6–10

FORMAL DEFINITION — A 5-TUPLE

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q = finite set of states · Σ = input alphabet · δ = transition function · $q_0 \in Q$ = start state · $F \subseteq Q$ = final / accepting states



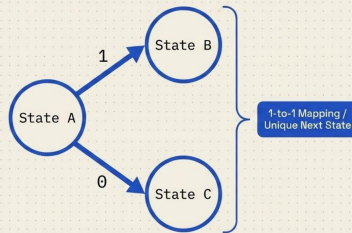
All five parts labelled on one machine (from the *NFA_Formal_Architecture* deck).

Anatomy of a Deterministic Transition

On getting input '1', State A goes to State B and no other states.

On getting input '0', State A goes to State C and no other states.

Design Constraint: Simple and easy to design due to zero choices and zero randomness.

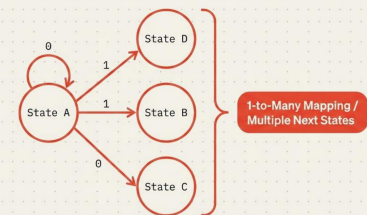


DFA: one input leads to exactly ONE next state.

Anatomy of a Non-Deterministic Transition

State A on getting input 0 could stay in State A or go to State C.

State A on getting input '1' could go to State B or go to State D.



NFA: one input can lead to MANY next states.

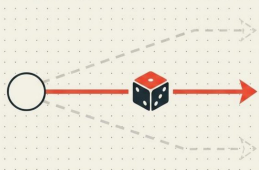
DFA vs NFA — the comparison table

	DFA (Deterministic)	NFA (Non-deterministic)
Transition function	$\delta : Q \times \Sigma \rightarrow Q$	$\delta : Q \times \Sigma \rightarrow 2^Q$ (power set). With ϵ -moves: $Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
Next state	exactly one, unique (“absolute assurance”)	zero, one or many
Missing transitions	✗ illegal — every state needs a move on every symbol	✓ allowed — that path dies ($\emptyset$)
ϵ (empty) moves	✗ never	✓ allowed (jump without reading input)
Acceptance	the single path ends in F	at least one path ends in F
Execution	one path	random choice or all paths in parallel
Design	rigid, may need more states (dead states)	easier, only draw the paths you need
Power	Equal — both accept exactly the regular languages; every NFA can be converted to a DFA	

The Mechanics of Choice: NFA Execution Models

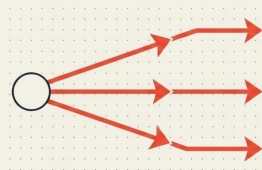
Random Selection

The system may choose the next state purely at random among the available valid pathways.



Parallel Execution

The system may choose all possible next states simultaneously, branching its execution across all valid pathways at once.



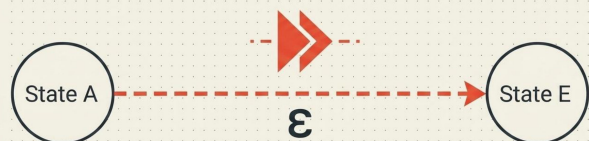
How an NFA “chooses”: at random, or in parallel.

The Epsilon Jump: Transitions Without Tokens

The Epsilon symbol represents the empty string, ϵ .

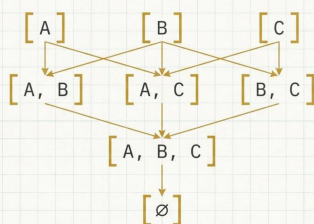
An NFA can accept an empty input to transition, allowing the machine to jump to a new state without consuming a character.

This concept exists strictly in NFA architectures, never in DFA.



ϵ -transition: a state change without consuming input (NFA only).

The 3-State Universe (A, B, C)



3 States = 8 Possibilities (2^3)

3 states $\rightarrow 2^3 = 8$ possible target sets. That is why the NFA's δ maps into 2^Q .

WHY 2^Q ?

From state q on symbol a , an NFA can go to *any subset* of Q : $\{A\}$, $\{B\}$, $\{A,B\}$, or $\emptyset$. With $|Q| = n$ there are 2^n subsets, so δ maps into the **power set** 2^Q .

EXAM FACTS

- Number of DFA states after conversion: $1 \leq n \leq 2^m$ (m = number of NFA states).
- A DFA recognises a string in time **linear** in its length.
- **Dead / trap state**: a non-final state that loops to itself on every symbol, so the machine can never reach F.
- Day/month FSA (PPT-1) **overgenerates**: it accepts 37/00.

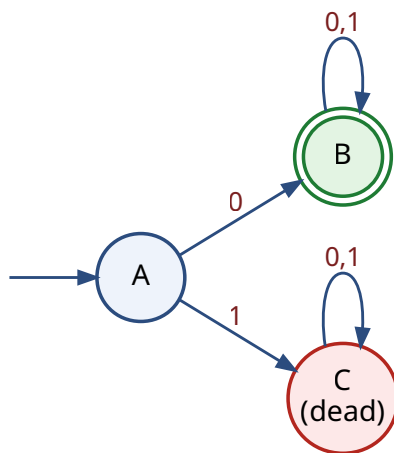
3B DFA Design Gallery & NFA → DFA

Lec 8–10

RECIPE FOR DESIGNING A DFA

1. Decide what each state must *remember* (prefix seen, count so far, remainder...). 2. Draw the start state. 3. Add a transition for **every symbol from every state**; add a dead state if needed. 4. Mark the final states. 5. Write the transition table and **trace 2 accepted + 2 rejected strings**.

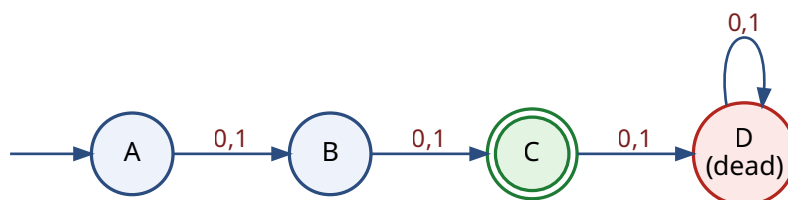
① Strings that start with 0 $\Sigma = \{0,1\}$



δ	0	1
$\rightarrow A$	B	C
*B	B	B
C	C	C

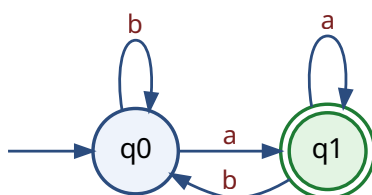
01 $\rightarrow A \rightarrow B \rightarrow B$ ✓ 101 $\rightarrow A \rightarrow C \rightarrow C \rightarrow C$ ✗ (trapped)

② Strings of exactly length 2

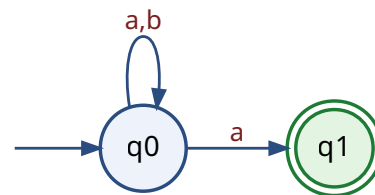


Each state counts characters: A = length 0, B = 1, **C = 2 (final)**, D = 3+ (dead).
 00, 10 ✓ · "1" ✗ (undershoot, stops at B) · 001 ✗ (overshoot, ends in D)

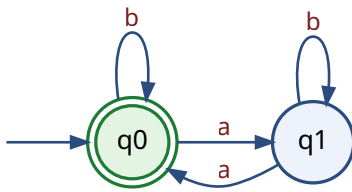
③ Ends with "a" — DFA



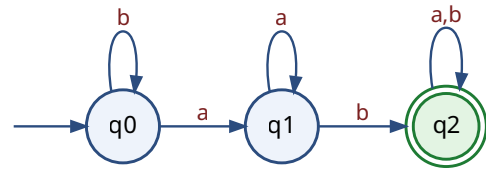
③ Ends with "a" — NFA



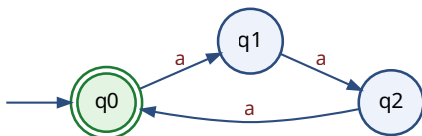
Same language, fewer arcs. q1 has no outgoing moves.

④ Even number of a's

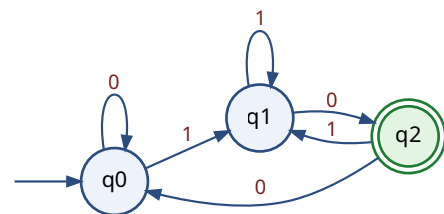
Regex: $(b \mid ab^*ab^*)^*$. For an **odd** number of a's, make q1 final. Minimum = 2 states.

⑤ Contains substring "ab"

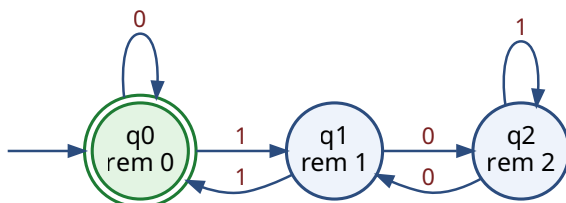
Regex: $(a \mid b)^*ab(a \mid b)^*$

⑥ Number of a's divisible by 3 (a^{3n})

For $3n+1$, make q1 final instead. In general, $\{a^{kn}\}$ needs k states.

⑦ Ends with "10" — regex $(0+1)^*10$ 

The minimal DFA has **3 states** (a classic MCQ).

⑧ Binary numbers divisible by 3

δ	0	1
$\rightarrow^*q0$	q0	q1
q1	q2	q0
q2	q1	q2

The state is the remainder so far. New remainder = $(2 \cdot r + \text{bit}) \bmod 3$.

1001 (= 9): $q0 \rightarrow q1 \rightarrow q2 \rightarrow q1 \rightarrow q0$ ✓

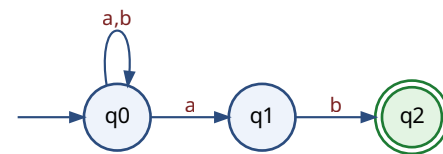
NFA → DFA by subset construction

1. Write the NFA transition table.
2. DFA start = $\{q_0\}$ (or its ε -closure).
3. For each new set S and symbol x : $\delta'(S, x) = \bigcup \delta(q, x), q \in S$.
4. Repeat until no new sets appear.
5. Any set that contains an NFA final state is a DFA final state.

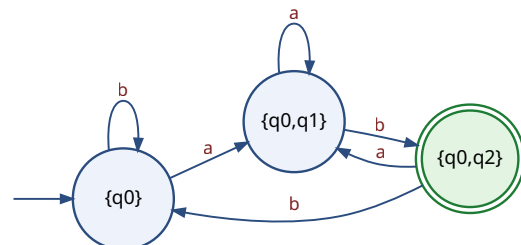
Example (strings ending in "ab"): $\delta(q_0, a) = \{q_0, q_1\}$,
 $\delta(q_0, b) = \{q_0\}$, $\delta(q_1, b) = \{q_2\}$, $F = \{q_2\}$

DFA state	a	b
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$* \{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

e.g. $\delta'(\{q_0, q_1\}, b) = \delta(q_0, b) \cup \delta(q_1, b) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$



NFA (3 states)



Equivalent DFA (3 states)

DFA minimisation (table method)

1. Remove states unreachable from q_0 .
2. Split states into *non-final* and *final* groups.
3. Within a group, merge rows with identical transitions (equivalent states).
4. Combine the groups to get the minimised DFA.

Equivalence of two FAs

Start with the pair $(start_1, start_2)$. For every input symbol, follow both machines. Each resulting pair must be **both final** or **both non-final**. Repeat for every new pair; if a mismatch appears, the machines are not equivalent.

4 Regular Expressions

Lec 10

A **regular expression** is a formal language for specifying text strings: a pattern describing the *shape* of text. In NLP it is used for tokenizing, cleaning and extracting information (emails, dates, phone numbers) and for validation. Every regex defines a **regular language**, so it can be converted (regex → NFA → DFA).

Disjunction, ranges, negation

Pattern	Matches
[wW]oodchuck	Woodchuck, woodchuck
[A-Z] [a-z] [0-9]	one upper / lower / digit
[^A-Z]	NOT an upper-case letter
[^Ss]	neither S nor s
[e^]	e or ^ (caret not first, so literal)
groundhog woodchuck	either word
a b c	same as [abc]

Shorthand classes

Alias	Same as	Meaning
\d / \D	[0-9] / [^0-9]	digit / non-digit
\w / \W	[a-zA-Z0-9_] / [^\w]	word char / non-word
\s / \S	[\r\t\n\f] / [^\s]	whitespace / non-whitespace

Wildcards, optionality, repetition

Pattern	Meaning	Matches
beg.n	any one char	begin, begun, beg3n
colou?r	0 or 1	color, colour
to*	0 or more (Kleene *)	t, to, too, tooo
to+	1 or more (Kleene +)	to, too, tooo
\d{4}	exactly n	2026
\d{2,4}	n to m times	12, 123, 1234

Anchors & boundaries

Pattern	Meaning
^[A-Z]	line starts with a capital ("Palo Alto")
\.\$	line ends with a period ("The end.")
\bcat\b	whole word "cat" (not "category")

TRAP

^ means NOT only when it is the first character inside []. Inside brackets, . * + ? lose their special meaning.

Writing a regex is iterative: false positives vs false negatives

Attempt	Problem
the	misses "The" (false negative)
[tT]he	matches "other", "Theology" (false positive)
\W[tT]he\W or \b[tT]he\b	✓ whole word only

PRECISION vs RECALL

False positive = matched something wrong.

Fixing it increases **accuracy / precision**.

False negative = missed something right.

Fixing it increases **coverage / recall**.

The two efforts are *antagonistic*: improving one often hurts the other.

Substitutions, capture groups & ELIZA

```
s/colour/color/
substitution
s/([0-9]+)/<\1>/          the 35 boxes
→ the <35> boxes
/the (.*)er they (.*) , the \1er we \2/
  ✓ the faster they ran, the faster we
  ran
  ✗ the faster they ran, the faster we
  ate
/(?:some|a few) (people|cats) like some
\1/      non-capturing
/^(?!Volcano)[A-Za-z]+/      negative
lookahead
```

() groups and captures into register \1, \2...; (?:) groups without capturing; (?=) / (?!) are zero-width lookaheads.

ELIZA (Weizenbaum, 1966)

An early chatbot that imitated a Rogerian psychotherapist using **only pattern matching + substitution**, with no understanding:

```
s/. * I'M (depressed|sad) .*/I AM SORRY TO
HEAR YOU ARE \1/
s/. * all .*/IN WHAT WAY?/
s/. * always .*/CAN YOU THINK OF A SPECIFIC
EXAMPLE?/
```

Common NLP patterns

Task	Pattern
Email	<code>[\w. -]+@[\w. -]+\. \w+</code>
URL	<code>https?:\/\/\S+</code>
Hashtag	<code>#\w+</code>
Phone	<code>\d{3}-\d{3}-\d{4}</code>
Date	<code>\d{1,2}\/\d{1,2}\/\d{4}</code>
Number	<code>-?\d+\.\?\d*</code>
Strip HTML	<code>re.sub(r"<.*?>", "", t) (? = non-greedy)</code>
Tokenize	<code>\w+ [^\w\s] → isn 't</code>

Python `re` module

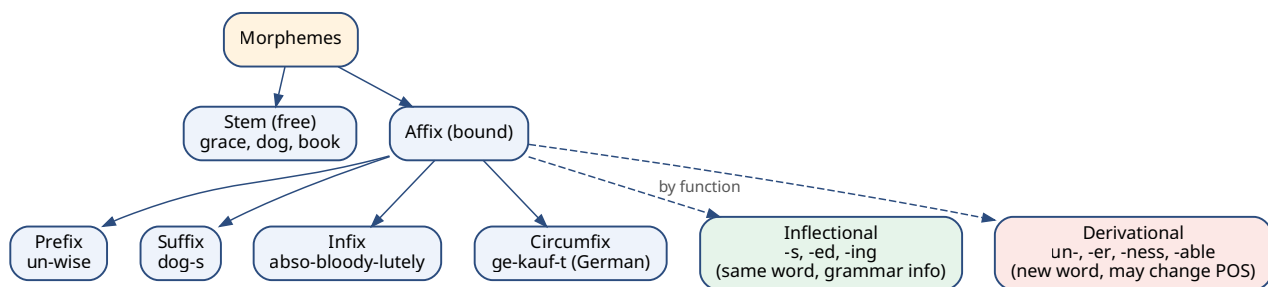
<code>re.match</code>	match only at the start of the string
<code>re.search</code>	first match anywhere
<code>re.findall</code>	list of all matches
<code>re.sub</code>	replace matches
<code>re.split</code>	split where the pattern matches

```
>>> re.findall(r'\d+', "I have 2 cats and 5 dogs")
['2', '5']
```

Always use raw strings: `r"\d+"`, otherwise you must write `"\\d+"`.

5 Morphology

Morphology is the study of word structure: how words are built from smaller meaningful pieces.

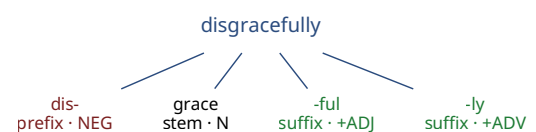


The vocabulary of morphology.

Term	Definition / example
Morpheme	smallest meaning-bearing (or grammatical) unit
Stem	core meaning unit, usually a free morpheme (can stand alone): grace
Affix	bound morpheme (cannot stand alone): dis-, -ful, -ly
Compound	more than one stem: book+shop+s, ice cream cone
Allomorphs	different surface forms (<i>morphs</i>) of one underlying morpheme: plural -s / -es / -ren
Paradigm	all inflected forms of one stem
Lemma vs wordform	cat & cats: same lemma, different wordforms

TRAP

s/- in slither/slide/slip shares a meaning but is **not** a morpheme.



Morphological parse of "disgracefully": prefix-stem-suffix-suffix.

AFFIX TYPES (most → least common)

Suffix: dog+s, truth+ful

Prefix: un+wise

Infix: abso-bloody-lutely

Circumfix: German ge+kauf+t (not in English)

Inflectional vs Derivational morphology

	Inflectional	Derivational
Purpose	adds grammatical info: tense, number, person, gender, case, aspect	creates a new word / new meaning
Part of speech	never changes	often changes: help (V) → helper (N)
Result	a form of the <i>same</i> lemma	a <i>different</i> lemma (new dictionary entry)
Productivity	applies to almost every stem	selective, but can stack: anti-anti-dis-establish-ment-arian-ism
English examples	-s (plural), -s (3sg), -ed (past), -ed/-en (participle), -ing	un-, re-, anti-, dis-, mis-; -ation, -er, -ness, -able, -al, -ism, -ize

English inflection

Verb form	Regular	Irregular
Base / present	walk	go
3sg (-s form)	walks	goes
Past	walked	went
Past participle	walked	gone
-ing form	walking	going

Nouns inflect for number (book/books). Pronouns inflect for person, number, gender and case (I/me, he/him). Irregular forms must be **listed**: sing-sang-sung, good-better-best.

English derivation

Process	Example
V + -ation → N	computerization
V + -er → N	killer, helper
Adj + -ness → N	fuzziness
un-, mis- (negation)	undo, mistake
V + -able → Adj	doable
N + -al → Adj	national
Zero derivation	tango, waltz (N → V)

Word-formation processes

- **Inflection** – forms of one word: book → books
- **Derivation** – grace → disgrace → disgraceful → disgracefully
- **Compounding** – cream → ice cream → ice cream cone → ice cream cone bakery
- All are **productive**: Google → Googler, to google, ungoogle, googlification...

Language typology

Type	Description	Example
Isolating	~1 morpheme per word	Yoruba
Agglutinative	many affixes, each carrying ONE piece of info	Turkish, Tamil, Telugu
Inflected (fusional)	one affix carries SEVERAL pieces of info	French
Analytic	little inflection; relies on word order and helper words	English

Agglutinative and inflected are both *synthetic* (many morphemes per word). English is not isolating, because it has derivational morphology.

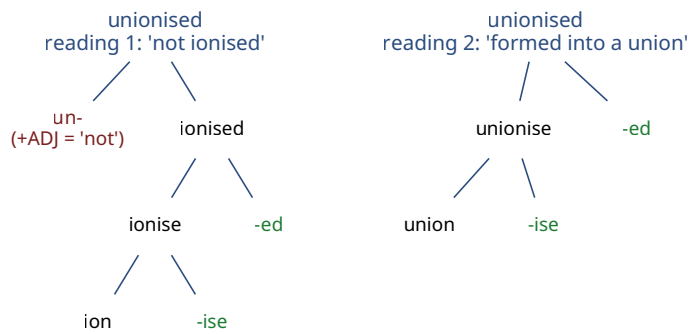
A Turkish word (agglutinative)

uygar·laş·tır·ama·dık·lar·ımız·dan·mı·ş·sınız·casına
 “as if you are among those whom we were not able to civilize”

uygar	civilized	-lar	plural
-laş	become	-ımız	our (1pl poss.)
-tır	cause to	-dan	among (ablative)
-ama	not able	-mı·ş	past
-dık	past participle	-sınız	you (2pl)
		-casına	as if

Morphological ambiguity

- **Morpheme ambiguity:** *paint* (N or V); +s (plural or 3sg verb)
- **Structural ambiguity:** *shorts* or *short* + *s*; blackberry vs cranberry
- **Bracketing:** *unionised* = union-ise-ed or un-ion-ise-ed. *un-ion* is not a word, so it must be **(un-((ion-ise)-ed))**.
- *un-* is itself ambiguous: with verbs it means **reversal** (untie); with adjectives it means **not** (unwise).



Two readings of "unionised".

LIMITATION

Finite-state methods (FSAs/FSTs) **cannot** represent this internal bracketing. Hierarchical structure such as (((ice cream) cone) bakery) needs a **CFG**.

Uses of morphological processing in NLP

- Compiling a full-form lexicon
- Stemming for information retrieval
- Lemmatization before parsing (morphosyntax)
- Generation (bidirectional)
- **Parsing:** parties → party + PLURAL
- **Generation:** sleep + PAST_VERB → slept

6 Finite-State Morphological Parsing & FSTs Lec 14-18

Morphological parsing maps a *surface form* to a *lexical form* (stem + features):

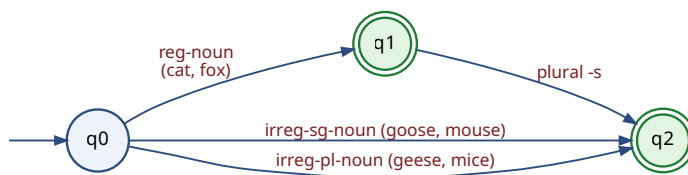
cats	→	cat +N +PL
foxes	→	fox +N +PL
disgracefully	→	dis(NEG) grace +N +ADJ +ADV

Generation is the reverse. The goal is to accept every possible English word (grace, graceful, ungracefully, undisgracefully...) and **reject** impossible ones (*gracefully, *disungracefully). Linguists mark ungrammatical forms with *.

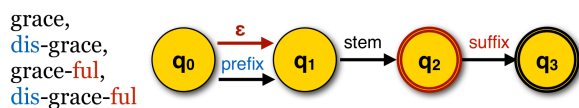
3 THINGS A MORPHOLOGICAL PARSER NEEDS

1. **Lexicon:** stems and affixes with their information (ed → PAST_VERB; s → PLURAL_NOUN; began → PAST_VERB begin)
2. **Morphotactics:** rules for which morphemes may follow which (an FSA)
3. **Orthographic rules:** spelling changes at morpheme boundaries (an FST)

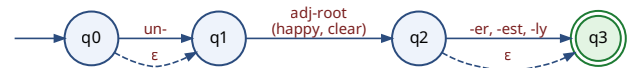
Building a finite-state lexicon (morphotactics)



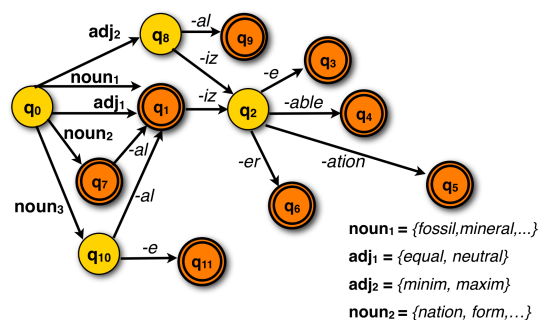
FSA for English noun inflection (J&M style).



Union with an ϵ -arc accepts *grace*, *dis-grace*, *grace-ful*, *dis-grace-ful* (PPT-2).



FSA for adjectives: (un-) + root + (-er/-est/-ly).



FSA for derivational morphology: *fossil* → *fossilize* → *fossilization*... (PPT-2).

Finite-State Transducers (FST)

DEFINITION — A 7-TUPLE

$T = \langle Q, \Sigma, \Delta, q_0, F, \delta, \sigma \rangle$

An FSA plus an **output alphabet** Δ (e.g. +N, +PL) and an **output function** $\sigma : Q \times \Sigma \rightarrow \Delta^*$.

Each arc is labelled **input : output**, e.g. $s:s, \epsilon:\wedge, \wedge:e$.

An FSA only *recognises*; an FST *analyses*, mapping the input to an output that encodes its structure. An FST defines a **relation between two regular languages**:

$T = \{ \langle \text{cat}, \text{cat+N+sg} \rangle, \langle \text{cats}, \text{cat+N+pl} \rangle, \langle \text{fox}, \text{fox+N+sg} \rangle, \langle \text{foxes}, \text{fox+N+pl} \rangle \}$

FST operation

What it does

Inversion T^{-1}

swaps input and output labels, so a parser becomes a generator

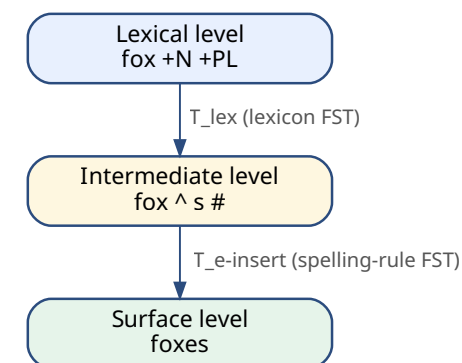
Composition $T \circ T'$ (cascade)

$T = L1 \times L2$ and $T' = L2 \times L3$ combine into $L1 \times L3$

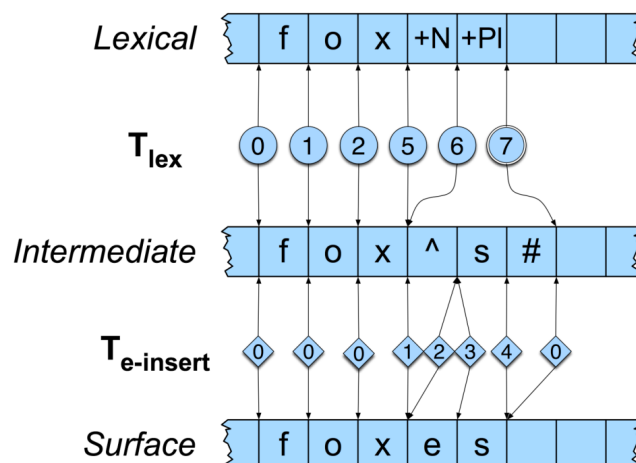
AMBIGUITY

book $\rightarrow$ book+N+sg or book+V. Analysis needs a **non-deterministic FST** plus a scoring function or context ("I read a book" vs "I book flights"). Not every NFST can be determinised. Generation is usually unambiguous.

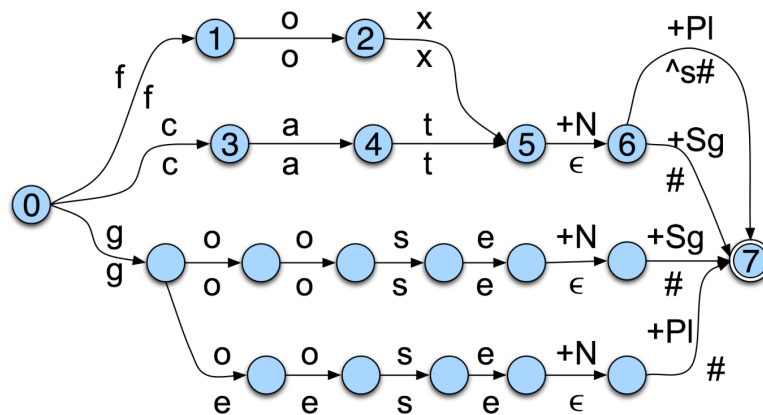
The two-level cascade: lexical $\rightarrow$ intermediate $\rightarrow$ surface



$\wedge$ = morpheme boundary · # = word boundary · ϵ = empty string



The cascade for fox+N+PL $\rightarrow$ foxes (PPT-2, Hockenmaier).



T_{lex} : lexical $\rightarrow$ intermediate. Regular nouns (fox, cat) get ^s#; irregular goose/geese are listed (PPT-2).

Orthographic (spelling) rules

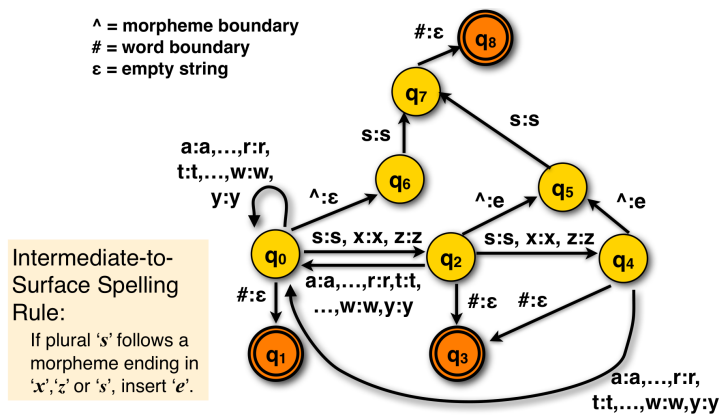
Rule	Description	Example
E-insertion	insert e after s, x, z (and ch, sh) before the -s suffix	fox^s $\rightarrow$ foxes, box^s $\rightarrow$ boxes
E-deletion	drop silent e before -ing / -ed	make^ing $\rightarrow$ making
Consonant doubling*	double a single final consonant before -ing / -ed	beg^ing $\rightarrow$ begging
Y-replacement*	-y becomes -ie before -s, and -i before -ed	try^s $\rightarrow$ tries, fly $\rightarrow$ flies
K-insertion*	verbs ending in vowel + c add k	panic^ed $\rightarrow$ panicked

* From the textbook (Jurafsky & Martin, Ch. 3). The slides show E-insertion and E-deletion.

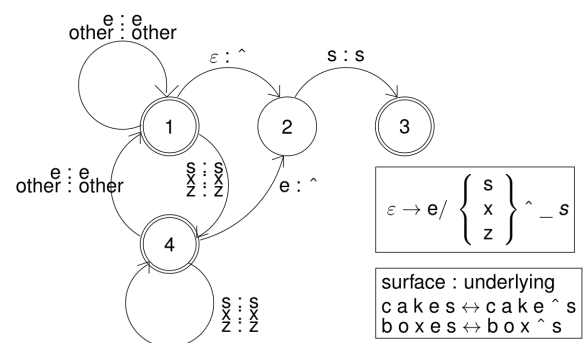
EXAM — THE E-INSERTION RULE (memorise this notation)

$$\epsilon \rightarrow e / \{s, x, z\} \wedge _ s$$

“Insert an e (rewrite ϵ as e) when a morpheme ending in s, x or z is followed by the affix boundary $\wedge$ and then s.” The mapping is left of the slash and the context is right of it; $_$ marks where the change happens. The same rule handles plural nouns and 3sg verbs, and it is implemented as an FST.



$T_{e\text{-insert}}$: intermediate $\rightarrow$ surface (PPT-2).



The same rule as a compact FST (PPT-1, Cambridge).

Tracing “boxes” in the Cambridge FST

b, o loop on state 1 (other:other); x:x moves 1 $\rightarrow$ 4.

At **e** there are two paths: e:Λ (4 $\rightarrow$ 2, output so far box^Λ) or e:e (4 $\rightarrow$ 1, output boxe).

Then **s**: s:s 2 $\rightarrow$ 3 ✓ gives **box^Λs**; on the other path 1 $\rightarrow$ 4 ✓ gives boxes. (A third path via ε:Λ gives boxe^Λs.)

The analyser keeps all three outputs; lexicon lookup then keeps **box^Λs**. Similarly cakes $\leftrightarrow$ cake^Λs.

Using FSTs (PPT-1)

- FSTs assume text is already tokenized and split into characters, with **one character pair per transition**.
- Analysis returns the character list with affix boundaries, which enables lexicon lookup.
- Generation takes its input from the stem and affix lexicons.
- Use **one FST per spelling rule**, then either compile them into one big FST or run them in parallel.
- Cannot model internal bracketing (un-ion-ize-d) or condition on earlier transitions.

Lexicon-free FST: the Porter stemmer



A **cascade of rewrite rules** with **no lexicon**; each pass feeds the next.

ATIONAL $\rightarrow$ ATE relational $\rightarrow$ relate

ING $\rightarrow$ ε if the stem has a vowel: motoring $\rightarrow$ motor

SSSES $\rightarrow$ SS grasses $\rightarrow$ grass

Output may be a non-word: “Thi wa not the map... Billi Bone... accur copi”.

Stemming vs Lemmatization

Stemming	Lemmatization
Crudely chops affixes using rules	Maps to the dictionary headword (lemma)
Output may not be a word: studies → studi , happiness → happi	Always a real word: studies → study , better → good , am/is/are → be
Ignores context and POS; fast, simple	Uses POS tagging + dictionary + morphological parsing; slower, accurate
Porter, Snowball, Lancaster; good for search indexing	Better for MT, QA, sentiment, topic modelling

Lemmatization is done by morphological parsing: “He is reading detective stories” → “He be read detective story”; Spanish *amaren* → amar + 3PL + future subjunctive.

7 Text Processing & Tokenization

Words, types, tokens and corpora

- **Type** = an element of the vocabulary (a distinct word).
- **Token** = one occurrence of a type in running text.
- **N** = number of tokens; **V** = vocabulary (set of types); $|V|$ = its size.
- **Lemma** = same stem, POS and rough sense (cat/cats). **Wordform** = full inflected surface form (cat ≠ cats).
- Speech adds **fragments** ("main- mainly") and **filled pauses** ("uh").

EXAM — COUNT THEM

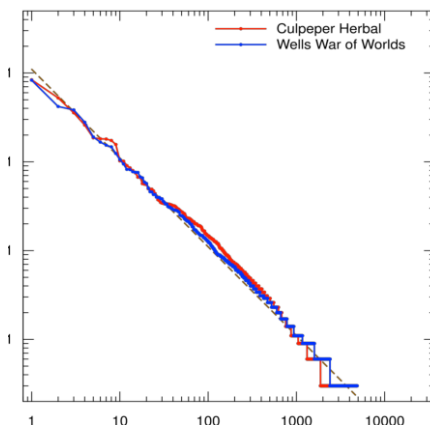
"they lay back on the San Francisco grass and looked at the stars and their" has **15 tokens** (or 14 if San Francisco counts as one) and **13 types** (or 12 or 11, depending on whether San Francisco is one word and whether the/ their are merged). Repeated words: the ×3, and ×2.

Corpus	Tokens N	Types $ V $
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
COCA	440 million	2 million
Google N-grams	1 trillion	13+ million

HEAPS' LAW (HERDAN'S LAW)

$$|V| = k \cdot N^\beta, \quad 0.67 < \beta < 0.75$$

Vocabulary grows faster than the square root of the number of tokens.



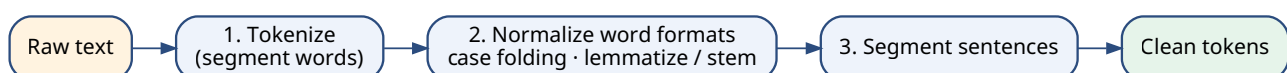
Zipf's law on a log-log plot (PPT-3).

ZIPF'S LAW

A word's frequency is **inversely proportional to its rank** ($f \propto 1/r$). A few words are very frequent (the, of, and) and there is a **long tail** of rare words, which is why word-level vocabularies explode and OOV words are unavoidable.

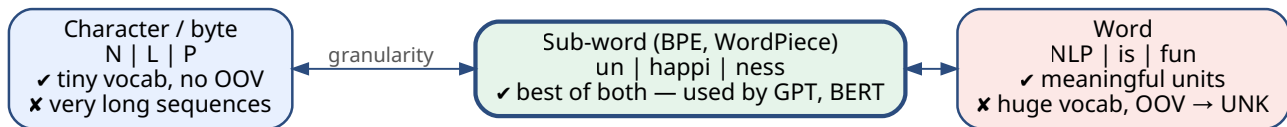
Corpora vary by language (≈ 7097 languages), variety (e.g. AAE "iont" = I don't), **code-switching** (Hindi/English: "dost tha or ra- hega ... dont worry"), genre and author demographics. **Datasheets** document a corpus's motivation, situation, collection process, annotation and demographics (Geburu et al. 2020).

Text normalization: 3 steps



Tokenization

Tokenization breaks text into smaller meaningful units called **tokens**. It is usually the **first preprocessing step**. A **tokenizer** translates between text and a sequence of token IDs; the **vocabulary** is the set of known tokens.



The granularity spectrum of tokenization.

Type	Example	Pros	Cons / challenges
Word	"NLP is fun, isn't it?" → NLP is fun , is n't it ?	meaningful units; one embedding $\approx$ one meaning	huge vocabulary (Zipf), OOV words → UNK; contractions, hyphens (state-of-the-art), MWEs (New York)
Sentence	"NLP is fun. It is used in AI." → 2 sentences	needed for parsing, MT, summarization	abbreviations (Dr., U.S.A.), decimals (3.14), ellipsis, quotes
Character / byte	NLP → N L P	tiny vocabulary, no OOV , sees spelling, language-independent	very long sequences, little meaning per token, costly
Sub-word	unhappiness → un happi ness	handles OOV, smaller vocabulary, captures morphology; used by BERT/GPT	must train a tokenizer; less interpretable
N-gram	NLP is fun → bigrams "NLP is", "is fun"	captures local word order	sparsity, large feature space

Issues in tokenization

- Can't blindly remove punctuation: **m.p.h., Ph.D., AT&T, cap'n, \$45.55, 01/02/06, http://www.stanford.edu, #nlproc, someone@cs.colorado.edu**
- **Clitics** are words that don't stand alone: 're in *we're*; French *j'* in *j'ai*, *l'* in *l'honneur*
- **Multi-word expressions**: New York, rock 'n' roll. One token or two?
- Bytes ≠ characters: an emoji is 1 character but 4 bytes; is 1 character but 3 bytes

Simple UNIX tokenizer ("UNIX for Poets", Ken Church)

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | uniq -c
```

tr turns every non-letter into a newline (one token per line), sort groups identical tokens, and uniq -c counts each type.

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

Case-fold first, then rank by count: 23243 the · 22225 i · 18618 and · 16339 to ...

Languages without spaces

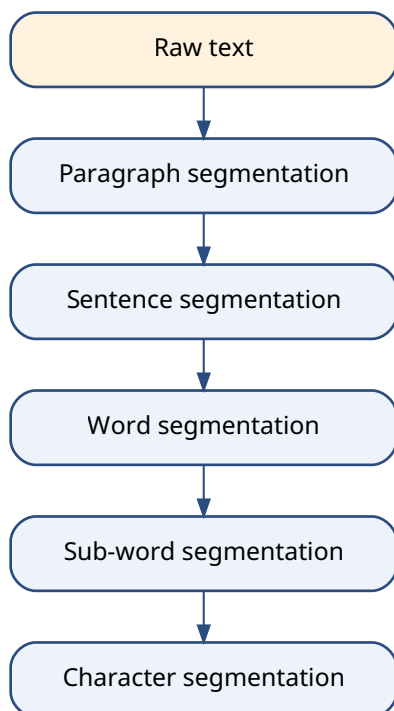
Chinese, Japanese and Thai don't separate words with spaces. A Chinese word is made of characters (**hanzi**), about 2.4 characters per word on average, each a morpheme. Chinese NLP often treats **each character as a token**. Thai and Japanese need neural sequence models for segmentation.

姚明进入总决赛 "Yao Ming reaches the finals"

姚明		进入		总决赛	
YaoMing		reaches		finals	
姚	明	进	入	总	决赛
姚	明	进	入	总	决 赛

3 words · 5 words · or 7 characters?

Levels of text segmentation



Sentence segmentation (sentence boundary detection)

- **!** and **?** are mostly unambiguous, but the **period “.”** is **ambiguous**: it can end a sentence, mark an abbreviation (Inc., Dr.), or appear in a number (.02%, 4.3).
- **Algorithm**: tokenize first, then use rules or ML to classify each period as *part of a word* or a *sentence boundary*. An **abbreviation dictionary** helps.
- Other hard cases: initials (A.P.J. Abdul Kalam), URLs, quotes, informal text.

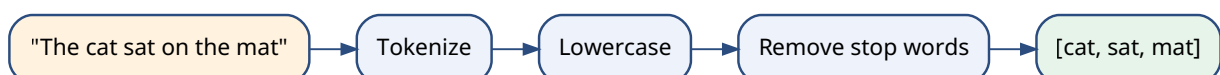
```

from nltk.tokenize import sent_tokenize
sent_tokenize("NLP is a part of AI. Dr. Rao teaches NLP
               at 10.30 a.m. It is useful!")
→ ['NLP is a part of AI.',
    'Dr. Rao teaches NLP at 10.30 a.m.', 'It is useful!']
  
```

Word normalization & case folding

Put tokens in a standard format: U.S.A./USA, uh-huh/uhhuh, Fed/fed, am/is/are → be. **Case folding** (lowercase everything) helps IR, but keep case for sentiment, MT and IE, because **US vs us** and **General Motors** matter.

Stop words



WHY REMOVE THEM

Stop words (the, is, at, of, and) are very frequent but carry little meaning.

- Less noise
- Smaller vocabulary: 3 reviews go from 24 to 14 unique words (≈40% fewer)
- Faster processing
- Better search: “What is the capital of France?” → capital, france

WHEN NOT TO REMOVE THEM

- **Sentiment**: “not good” → “good” flips the meaning
 - Idioms: “to be or not to be”
 - Machine translation (word order and function words matter)
 - **QA**: who / what / where carry the intent
- There is no universal list; tailor it by language and domain (news: said, according, reuters).

Tools: `nltk.corpus.stopwords.words('english')`, `spaCy token.is_stop`, `sklearn ENGLISH_STOP_WORDS`.
 Pitfalls: removing negations, case-sensitivity bugs (lowercase first), using the wrong language's list.

8 Byte Pair Encoding (BPE)

Lec 20

Instead of splitting on whitespace or on single characters, **let the data decide** how to tokenize, giving **subword tokenization**. Three common algorithms:

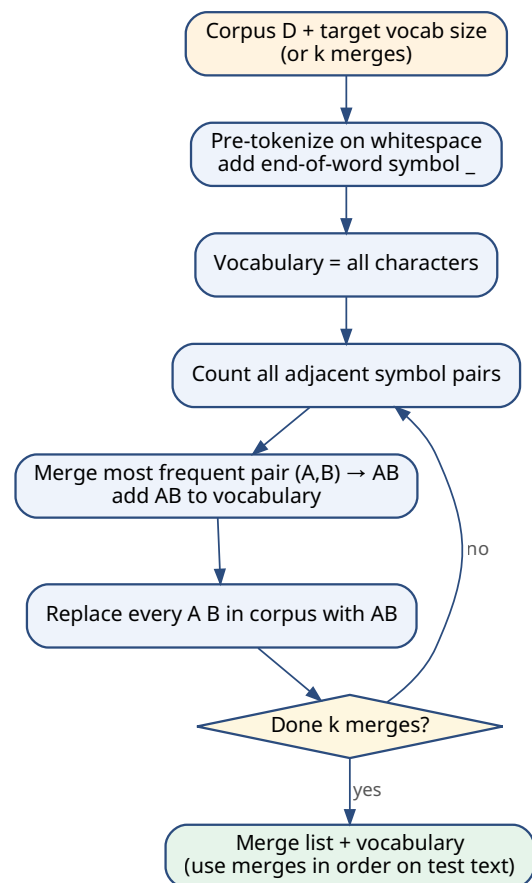
Algorithm	Idea
BPE (Sennrich 2016)	start from characters/bytes; repeatedly merge the most frequent adjacent pair
WordPiece (Schuster & Nakajima 2012; Wu 2016)	like BPE, but merge the pair that most increases the likelihood of the data
Unigram LM (Kudo 2018)	start with a large vocabulary and discard tokens until the target size

Each has two parts:

- **Token learner:** reads the training corpus and induces the vocabulary.
- **Token segmenter:** tokenizes new text using that vocabulary.

WHY BPE?

Used by almost all modern LLMs (GPT-4o/5 ≈ 200k vocab, Llama 3 ≈ 128k, Gemini ≈ 262k). Popularised by GPT-2. Simple, efficient, deterministic; with byte-level BPE nothing is OOV, and related words share pieces (picked/picking).



The BPE token-learner algorithm.

Worked example (the J&M corpus — practise this!)

Corpus: low ×5 · lowest ×2 · newer ×6 · wider ×3 · new ×2 → add end-of-word _ **Initial vocabulary:** _
d e i l n o r s t w

#	Merge	Freq	Corpus after merge
1	e + r → er	9	l o w _ ×5 l o w e s t _ ×2 n e w er _ ×6 w i d er _ ×3 n e w _ ×2
2	er + _ → er_	9	l o w _ ×5 l o w e s t _ ×2 n e w er_ ×6 w i d er_ ×3 n e w _ ×2
3	n + e → ne	8	l o w _ ×5 l o w e s t _ ×2 ne w er _ ×6 w i d er _ ×3 ne w _ ×2
4	ne + w → new	8	l o w _ ×5 l o w e s t _ ×2 new er _ ×6 w i d er _ ×3 new _ ×2
5	l + o → lo	7	lo w _ ×5 lo w e s t _ ×2 new er _ ×6 w i d er _ ×3 new _ ×2
6	lo + w → low	7	low _ ×5 low e s t _ ×2 new er _ ×6 w i d er _ ×3 new _ ×2
7	new + er_ → newer_	6	low _ ×5 low e s t _ ×2 newer_ ×6 w i d er _ ×3 new _ ×2
8	low + _ → low_	5	low_ ×5 low e s t _ ×2 newer_ ×6 w i d er _ ×3 new _ ×2

Final vocabulary: _ d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

HOW THE COUNTS ARE COMPUTED

Count each adjacent pair **weighted by word frequency**. Example: **e r** appears in newer (×6) and wider (×3), so its count is **9**. **r _** also has 9; ties are broken by order (the slides merge e r first). After the merges the corpus is

low_ ×5 · low e s t _ ×2 · newer_ ×6 · w i d er _ ×3 · new _ ×2.

TOKEN SEGMENTER (TEST TIME)

Apply the learned merges **greedily, in the order they were learned**. Test-set frequencies play no role.

- “n e w e r _” → **newer_** (1 token)
- “l o w e r _” → **low er_** (2 tokens)

Second walkthrough (PPT-3): “Peter Piper picked a peck of pickled peppers”

#	Merge	Count
1	_ + p → _p	4
2	c + k → ck	3
3	e + r → er	3
4	_p + e → _pe	2
5	_p + i → _pi	2
6	_pi + ck → _pick	2

Segment the new word “_pier”:

```

bytes:      _ p i e r
merge 1:    _p i e r
merge 3:    _p i er
merge 5:    _pi er      ← final tokens

```

(The merges that don't apply are skipped.)

Properties and problems of subword tokenization

- BPE tokens include frequent words *and* frequent subwords, often **morphemes** such as *-est*, *-er*.
Example: *unlikeliest* = *un-* + *likely* + *-est*.
- Segmentations are often **not linguistically aligned** (“*Milky*”, “*looked*”).
- **Spelling problem**: *strawberry* → [*str*, *aw*, *berry*], so the model can’t easily count the *r*’s, spell or reverse words, and is fragile to typos.
- **Glitch tokens**: when tokenizer and LM training data are mismatched, some tokens are barely trained (“*SolidGoldMagikarp*”).
- **SentencePiece** is a *library* (not an algorithm) implementing BPE and Unigram; it treats whitespace as a symbol (T5, Llama 2, XLM-R). **SuperBPE** learns superword tokens.

We 'd sat back on the grass , and
time flew by as we looked at the
Milky Way .

ChatGPT’s tokenizer: each colour is one token (from PPT-3). Note “We’d” → We + ‘d, and the leading space is part of the token (“*looked*”).

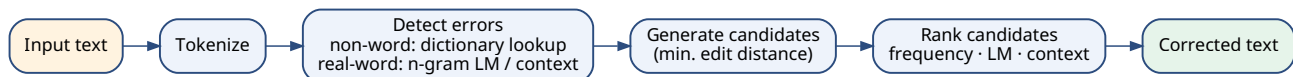
Language	Tokenization	# Tokens	# Characters
English	We'd sat back on the grass, and time flew by as we looked at the Milky Way.	21	75
French	Nous étions assis dans l'herbe, et le temps a filé tandis que nous contemplions la Voie lactée.	27	95
Somali	Waxaan dib u fadhiisanay cawskii, wakhtiguna wuu duulay markii aanu eegin Jidka caanaha.	30	88
Thai	พวกเรานั่งพักผ่อนบนพุ่มหญ้า และเวลาผ่านไปอย่างรวดเร็วขณะที่เรามองดูทางช้างเผือก	36	79

The same sentence costs more tokens in other languages (PPT-3).

MULTILINGUALITY (PPT-3)

- About 7000 languages exist, but 40–45% of Common Crawl is English. Languages are **high-resource** or **low-resource**.
- **Cross-lingual transfer**: fine-tune in English, then use zero-shot in other languages (XLM-R, mBERT).
- **Curse of multilinguality**: with fixed capacity, adding languages eventually hurts all of them (XNLI accuracy: 71.8% with 7 languages vs 67.7% with 100).
- **Subword fertility** = tokens per word. Higher fertility means worse performance and higher API cost (up to 15× length differences).

9 Spelling Errors & Minimum Edit Distance Lec 20 notes



End-to-end: "Ths is a smple sentnce." → detect Ths, smple, sentnce → "This is a simple sentence."

Non-word errors

The misspelling is **not in the dictionary**: recieve, mesage, languaeg, shcool.

Detect by dictionary lookup or character n-gram analysis.

Real-word errors

The word is valid but **wrong in context**: "I will **meat** you", "two/too books", "**there** house", "his **write** hand".

Dictionary lookup **can't** catch these; you need an n-gram LM or context.

Causes: typing mistakes, keyboard proximity, phonetic similarity, OCR errors, lack of language proficiency.

Correction method	How it works	Example
Dictionary-based	closest word in the lexicon; simple and fast, but can't fix real-word errors	mesage → message
Edit distance	choose the candidate with the minimum Levenshtein distance	spelng → spelling (1 insertion)
Phonetic	Soundex, Metaphone, Double Metaphone	nite → night
Rule-based	hand-written rules ("i before e except after c")	recieve → receive
Statistical LM	unigram/bigram/trigram probabilities in context	there house → their house
Machine learning	classification or ranking with features (edit distance, frequency, context); Naive Bayes, SVM, CRF	—
Neural	RNN / LSTM / Transformer; character + context patterns	modern autocorrect

Minimum Edit Distance (Levenshtein distance)

MED is the **minimum number of operations** needed to turn one string into another:

- **Insertion:** bok → book
- **Deletion:** books → book
- **Substitution:** cat → cut
- (Damerau adds **transposition**: recieve → receive)

Each operation costs 1. Some versions charge **2 for a substitution**; read the question.

DYNAMIC PROGRAMMING RECURRENCE

Table D of size $(m+1) \times (n+1)$: rows = source X, columns = target Y.

```

D[i][0] = i      D[0][j] = j
D[i][j] = min( D[i-1][j]  + 1,      ← deletion
               D[i][j-1]  + 1,      ← insertion
               D[i-1][j-1] + cost ) ← substitution
cost = 0 if  $x_i = y_j$ , else 1

```

The answer is the bottom-right cell **D[m][n]**. Trace back through the table to recover the alignment.

cat → cut = 1

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2	1	1	2
t	3	2	2	1

speling → spelling = 1

	ε	s	p	e	l	l	i	n	g
ε	0	1	2	3	4	5	6	7	8
s	1	0	1	2	3	4	5	6	7
p	2	1	0	1	2	3	4	5	6
e	3	2	1	0	1	2	3	4	5
l	4	3	2	1	0	1	2	3	4
i	5	4	3	2	1	1	1	2	3
n	6	5	4	3	2	2	2	1	2
g	7	6	5	4	3	3	3	2	1

kitten → sitting = 3

	ε	s	i	t	t	i	n	g
ε	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

k→s (sub), e→i (sub), +g (insert)

intention → execution = 5

	ε	e	x	e	c	u	t	i	o	n
ε	0	1	2	3	4	5	6	7	8	9
i	1	1	2	3	4	5	6	6	7	8
n	2	2	2	3	4	5	6	7	7	7
t	3	3	3	3	4	5	5	6	7	8
e	4	3	4	3	4	5	6	6	7	8
n	5	4	4	4	4	5	6	7	7	7
t	6	5	5	5	5	5	5	6	7	8
i	7	6	6	6	6	6	6	5	6	7
o	8	7	7	7	7	7	7	6	5	6
n	9	8	8	8	8	8	8	7	6	5

= 8 if substitution costs 2

Highlighted cells = one optimal back-trace path; the green cell = the answer.

Pair	sub = 1	sub = 2
cat → cut	1	2
kitten → sitting	3	5
intention → execution	5	8
speling → spelling	1	1
recieve → receive	2 (1 with transposition)	2
smple → simple / sample / smile	1 / 1 / 1	1 / 1 / 2

CORRECTION TO THE LECTURE-20 NOTES

The notes list *smple*: simple = 1, sample = 2, smile = 2. Under standard Levenshtein, **sample** = 1 (insert a) and **smile** = 1 (substitute p→i) too. “simple” wins because of **word frequency / context**, not distance alone. Write this reasoning if asked.

Applications of MED: spelling correction & autocorrect · OCR correction · speech recognition evaluation (**Word Error Rate**) · search query matching · plagiarism detection · DNA/protein sequence alignment · morphological variant matching.

10 Likely Exam Questions

1. Define NLP. Explain the phases/levels of NLP with an example of each.
2. What is ambiguity? Explain lexical, syntactic, semantic and pragmatic ambiguity with examples. How is it resolved?
3. List the challenges and applications of NLP.
4. Define alphabet, string, language, Σ^* , Σ^+ . How many strings of length n exist over $\{a,b\}$?
5. Differentiate an acceptor from a transducer.
6. Give the formal 5-tuple of a DFA and an NFA. Why does an NFA's δ map into 2^Q ?
7. Compare DFA and NFA (table, at least 6 points).
8. Design a DFA: starts with 0 / exactly length 2 / ends with "ab" / even a's / contains "ab" / divisible by 3. Give the transition table and trace strings.
9. Convert the given NFA to an equivalent DFA.
10. Minimum number of states of the DFA for $(0+1)^*10$. (Answer: 3)
11. Write regular expressions for: the word "the", email, phone number, date, a simple tokenizer.
12. Explain false positives vs false negatives, and precision vs recall.
13. Explain capture groups and substitution. How did ELIZA work?
14. Define morpheme, stem, affix, free vs bound morpheme, allomorph. Give the 4 affix types.
15. Differentiate inflectional and derivational morphology with English examples.
16. Explain isolating, agglutinative, inflected and analytic languages. Why is English analytic?
17. Explain morphological ambiguity with "unionised".
18. What is morphological parsing? What three components does it need?
19. Draw an FSA lexicon for English noun plurals / grace–disgrace–graceful.
20. Define an FST (7-tuple). Explain inversion and composition.
21. Explain the e-insertion rule $\varepsilon \rightarrow e / \{s,x,z\}^* _ s$ and trace "foxes / boxes" through the FST.
22. Explain the lexical $\rightarrow$ intermediate $\rightarrow$ surface cascade.
23. What is a lexicon-free FST? Explain the Porter stemmer.
24. Stemming vs lemmatization (with examples).
25. Define type and token; count them in a sentence. State Heaps' law and Zipf's law.
26. Explain word, sentence, character and subword tokenization and their challenges.
27. Why is tokenization hard for Chinese? Why is "." ambiguous in sentence segmentation?
28. Apply BPE to a small corpus for k merges; segment a new word.
29. What are stop words? When should they not be removed?
30. Non-word vs real-word spelling errors; methods of correction.
31. Compute the minimum edit distance with a DP table (e.g. intention $\rightarrow$ execution).

11 One-Page Revision Sheet

AUTOMATA

DFA = $(Q, \Sigma, \delta, q_0, F)$, $\delta: Q \times \Sigma \rightarrow Q$
 NFA $\delta: Q \times \Sigma \rightarrow 2^Q$ (ϵ -moves allowed)
 FST = $(Q, \Sigma, \Delta, q_0, F, \delta, \sigma)$, $\sigma: Q \times \Sigma \rightarrow \Delta^*$
 NFA $\rightarrow$ DFA: $1 \leq n \leq 2^m$ states
 DFA $\equiv$ NFA $\equiv$ regex = regular languages
 $a^n b^n$ is NOT regular (needs a PDA)
 Dead state = non-final, loops on everything
 $\{a^{kn}\}$ needs k states · $(0+1)^*10$ needs 3

REGEX

[] set · [^] not · . any · ? 0/1 · * 0+ · + 1+
 · {n,m}
 ^ \$ anchors · \b boundary · \d \w \s
 () capture, \1 back-ref · (?:) non-capture ·
 (?:) (?!) lookahead
 FP $\rightarrow$ precision; FN $\rightarrow$ recall

MORPHOLOGY

Morpheme = smallest meaningful unit
 Stem = free · affix = bound
 Prefix, suffix, infix, circumfix
 Inflectional = grammar, same word, no POS change
 Derivational = new word, may change POS
 Allomorph: -s/-es/-ren
 English = analytic; Turkish = agglutinative; French = inflected; Yoruba = isolating

FST & SPELLING RULES

fox+N+PL $\rightarrow$ fox[^]s# $\rightarrow$ foxes
 ^ morpheme boundary, # word boundary
 $\epsilon \rightarrow e / \{s,x,z\}^{\wedge} _ s$
 E-deletion: make+ing $\rightarrow$ making
 Inversion T^{-1} : parse $\leftrightarrow$ generate
 Composition $T \circ T'$: cascade
 Porter = lexicon-free FST cascade

TOKENIZATION

Type = distinct word · token = occurrence
 Heaps: $|V| = kN^{\beta}$, $\beta \in (0.67, 0.75)$
 Zipf: frequency $\propto 1/\text{rank}$
 Normalization: tokenize $\rightarrow$ normalize $\rightarrow$ segment sentences
 Word: OOV problem · Char: long sequences ·
 Subword: best of both
 "." is ambiguous (Dr., 4.3) · Chinese: treat characters as tokens

BPE

Learner: start from characters; merge the most frequent adjacent pair k times
 Segmenter: apply merges in learned order, greedily
 J&M merges: er, er_, ne, new, lo, low, newer_, low_, lower_ $\rightarrow$ low + er_
 Others: WordPiece (likelihood), Unigram (prune)

EDIT DISTANCE

$D[i][j] = \min(D[i-1][j]+1, D[i][j-1]+1, D[i-1][j-1]+\text{cost})$
 kitten $\rightarrow$ sitting = 3 · intention $\rightarrow$ execution = 5 (8 if sub = 2) · cat $\rightarrow$ cut = 1
 Non-word error $\rightarrow$ dictionary lookup
 Real-word error $\rightarrow$ language model / context

STEM vs LEMMA & STOP WORDS

studies $\rightarrow$ studi (stem) / study (lemma)
 better $\rightarrow$ good (lemma only)
 Stop words: the, is, at... remove for IR / BoW
 KEEP them for sentiment ("not"), MT, QA

Figure credits: slide figures are reproduced from the course materials in this repository (PPT-1: Buttery/Copestake, Cambridge; PPT-2: Hockenmaier, UIUC CS447; PPT-3: Kallini, Stanford CS224N; Beyond_Determinism / DFA / NFA decks; NLP-UNIT-1 notes; Dr. Sandeep Gupta's RE & tokenization slides). All other diagrams were drawn for these notes with Graphviz.